Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №6 «Итераторы, генераторы и декораторы»

по дисциплине «Программирование»

Вариант 5-2-2-2

Факультет: Безопасность информационных технологий

Группа: N3149

Выполнила: Беленькая Полина Александровна

Подпись:

Преподаватель: Гирик Алексей Валерьевич

Санкт-Петербург 2025

Задание

Разработать на языке Python два модуля и набор тестов для них:

- модуль, содержащий определение заданного вариантом из Табл. 1 и 2 генератора псевдослучайных чисел;
- модуль, содержащий определение декоратора, заданного вариантом из Табл. 3 и 4, позволяющего выполнить замену аргументов или возвращаемых значений функций/методов и логирование вызовов;
- набор тестов для pytest, позволяющий проверить корректность разработанных модулей.

Алгоритм генерации псевдослучайных чисел- Квадратичный конгруэнтный генератор (расширение ЛКГ)

Способ реализации ГПСЧ – генератор

Действие, выполняемое декоратором - @subst_float_args

Способ реализации декоратора – класс

Информация об алгоритме ГПСЧ и выбранных коэффициентах:

Формула для генерации: $x_{i+1} = (ax_i^2 + bx_i + c) \bmod M$

X_i - текущее значение (начальное значение задаётся seed)

a, b, c - коэффициенты, влияющие на "хаотичность" последовательности

m - модуль, определяющий диапазон значений ($0 \le x_i < m$)

В реализуемом квадратичном конгруэнтном генераторе значения коэффициентов a, b, c, m и начальное значение seed задаются вручную как аргументы.

Во время тестирования я использовала несколько разных наборов параметров, например: a = 2, b = 3, c = 1, m = 97; a = 1, b = 1, c = 1, m = 127; a = 3, b = 2, c = 1, m = 211. Это позволило проверить работу генератора в различных условиях и убедиться, что он работает правильно при разных первоначальных данных.

Реализация генератора:

```python
def quadratic_congruential_generator(
    seed: int,
    a: int,
    b: int,
    c: int,
    m: int
) -> Iterator[int]:
    x = seed
    while True:
        x = (a * x ** 2 + b * x + c) % m
        yield x
```

Проверка программы с pytest

```
polina@DESKTOP-QBMU2N6:~/prg6pabN3149$ pytest
========================================== test session starts ==========================================pla
y-1.4.0
rootdir: /home/polina/prg6pabN3149
collected 9 items

test_prg6pabN3149_rnd.py .....                                                                    [ 55%]
test_prg6pabN3149_subst.py ....                                                                   [100%]
```

Примеры работы программ на различных исходных данных:

1. Создание генератора с параметрами a=2, b=3, c=4, m=100 и генерация 10 случайных целых чисел методом next()

```
>>> from prg6pabN3149_rnd import quadratic_congruential_generator
>>> gen = quadratic_congruential_generator(seed=1, a=2, b=3, c=4, m=100)
>>> print([next(gen) for _ in range(10)])
[9, 93, 81, 69, 33, 81, 69, 33, 81, 69]
```

2. Создание объекта MyPRNG, получение 5 псевдослучайных вещественных чисел методом next_float()

```
>>> from prg6pabN3149_rnd import MyPRNG
>>> prng = MyPRNG(quadratic_congruential_generator(seed=5, a=2, b=3, c=1, m=100), m=100)
>>> print([prng.next_float() for _ in range(5)])
[0.66, 0.11, 0.76, 0.81, 0.66]
```

3. Генерация случайной строки длиной 10 символов методом next_str()

```
>>> prng = MyPRNG(quadratic_congruential_generator(seed=7, a=1, b=1, c=1, m=127), m=127)
>>> print(prng.next_str(10))
5fFQcbdn4s
```

4. Случайное перемешивание символов строки "abcdef" методом shuffle_str()

```
>>> prng = MyPRNG(quadratic_congruential_generator(seed=42, a=3, b=2, c=1, m=211), m=211)
>>> original = "abcdef"
>>> print(prng.shuffle_str(original))
cfbdea
```

5. Подмена аргументов типа float в функции calc_area(). В лог выводится информация о замене и итоговом результате.

```
polina@DESKTOP-QBMU2N6:~/prg6pabN3149$ python3
Python 3.12.3 (main, Feb  4 2025, 14:48:35) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from prg6pabN3149_rnd import MyPRNG, quadratic_congruential_generator
>>> from prg6pabN3149_subst import subst_float_args
>>> gen = quadratic_congruential_generator(seed=1, a=2, b=3, c=1, m=97)
>>> prng = MyPRNG(gen, m=97)
>>> @subst_float_args(prng)
... def calc_area(length: float, width: float) -> float:
...     return length * width
...
>>> print(calc_area(3.5, 2.0))
13.06.25 18:40:40 calc_area: аргумент 3.5 (length) заменён на 0.061855670103092786
13.06.25 18:40:40 calc_area: аргумент 2.0 (width) заменён на 0.9381443298969072
13.06.25 18:40:40 calc_area(length=3.5, width=2.0) -> 0.058029546179190136
0.058029546179190136
```

6. Декоратор применяется ко всему классу. Аргумент value: float заменяется при вызове, и лог отображает и подмену, и результат метода.

```
>>> from prg6pabN3149_rnd import MyPRNG, quadratic_congruential_generator
>>> from prg6pabN3149_subst import subst_float_args
>>> gen = quadratic_congruential_generator(seed=1, a=2, b=3, c=1, m=97)
>>> prng = MyPRNG(gen, m=97)
>>> @subst_float_args(prng)
... class Multiplier:
...     def __init__(self, factor: float):
...             self.factor = factor
...     def apply(self, value: float) -> float:
...             return self.factor * value
...
>>> obj = Multiplier(2.5)
>>> print(obj.apply(4.0))
13.06.25 18:56:23 apply: аргумент 4.0 (value) заменён на 0.061855670103092786
13.06.25 18:56:23 apply(self=<__main__.Multiplier object at 0x7adb5306f860>, value=4.0) -> 0.15463917525773196
0.15463917525773196
```

Исходный код программы:

prg6pabN3149_rnd.py

```python
from abc import ABC, abstractmethod
from collections.abc import Iterator
import string


# Генератор: Квадратичный конгруэнтный ГПСЧ
def quadratic_congruential_generator(
    seed: int,
    a: int,
    b: int,
    c: int,
    m: int
) -> Iterator[int]:
    x = seed
    while True:
        x = (a * x ** 2 + b * x + c) % m
        yield x

# Абстрактный базовый класс
class PRNGBase(ABC):
    @abstractmethod
    def next_int(self) -> int:
        """Возвращает случайное целое число"""
        pass

    @abstractmethod
    def next_float(self) -> float:
        """Возвращает случайное вещественное число в диапазоне от 0.0 до 1.0 (не включая)"""
        pass

    @abstractmethod
    def next_str(self) -> str:
        """Возвращает случайную последовательность символов"""
        pass

    @abstractmethod
    def shuffle_str(self, src: str) -> str:
        """Получает строку src и возвращает строку, содержащую символы из src, переставленные в случайном порядке"""
        pass

```

```python
# Реализация обёртки для генератора
class MyPRNG(PRNGBase):
    def __init__(self, gen: Iterator[int], m: int):
        self.gen = iter(gen)
        self.m = m

    def next_int(self) -> int:
        return next(self.gen)

    def next_float(self) -> float:
        return self.next_int() / self.m  # Нормализуем в диапазон от 0.0 до 1.0

    def next_str(self, length: int = 8) -> str:
        alphabet = string.ascii_letters + string.digits
        return ''.join(
            alphabet[self.next_int() % len(alphabet)]
            for _ in range(length)
        )

    def shuffle_str(self, src: str) -> str:
        chars = list(src)
        for i in range(len(chars) - 1, 0, -1):
            j = self.next_int() % (i + 1)
            chars[i], chars[j] = chars[j], chars[i]
        return ''.join(chars)

```

prg6pabN3149_subst.py

```python
from typing import Callable, Any, Optional
from functools import wraps
from datetime import datetime
import sys
import inspect

from prg6pabN3149_rnd import MyPRNG

class subst_float_args:
    def __init__(self, prng: MyPRNG, filename: Optional[str] = None):
        self.prng = prng
        self.filename = filename

    def __call__(self, obj: Any) -> Any:
        if inspect.isclass(obj):
            # Декорирование класса, заменяем все немагические методы
            for attr_name, attr in vars(obj).items():
                if callable(attr) and not attr_name.startswith("__"):
                    setattr(obj, attr_name, self._decorate(attr))
            return obj
        else:
            # Декорирование функции
            return self._decorate(obj)

    def _decorate(self, func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args, **kwargs):
            bound = inspect.signature(func).bind(*args, **kwargs)
            bound.apply_defaults()

            original_args = dict(bound.arguments)

            # Подменяем float аргументы
            for name, value in bound.arguments.items():
                if isinstance(value, float):
                    new_val = self.prng.next_float()
                    self._log_replacement(func.__name__, name, value, new_val)
                    bound.arguments[name] = new_val

            result = func(*bound.args, **bound.kwargs)

            self._log_call(func.__name__, original_args, result)
            return result

        return wrapper

    def _log_call(self, func_name: str, args: dict, result: Any) -> None:
        timestamp = datetime.now().strftime('%d.%m.%y %H:%M:%S')
        args_str = ', '.join(f"{k}={v!r}" for k, v in args.items())
        msg = f"{timestamp} {func_name}({args_str}) -> {result!r}"
        self._write_log(msg)

    def _log_replacement(self, func_name: str, arg_name: str, old: Any, new: Any) -> None:
        timestamp = datetime.now().strftime('%d.%m.%y %H:%M:%S')
        msg = f"{timestamp} {func_name}: аргумент {old!r} ({arg_name}) заменён на {new!r}"
        self._write_log(msg)

    def _write_log(self, msg: str) -> None:
        if self.filename:
            with open(self.filename, 'a', encoding='utf-8') as f:
                f.write(msg + '\n')
        else:
            print(msg, file=sys.stderr)
```

test_prg6pabN3149_rnd.py

```python
from prg6pabN3149_rnd import MyPRNG, quadratic_congruential_generator

# Проверка, что генератор выдает целые числа в нужном диапазоне
def test_1():
    gen = quadratic_congruential_generator(seed=1, a=2, b=3, c=4, m=97)
    for _ in range(10):
        val = next(gen)
        assert isinstance(val, int)
        assert 0 <= val < 97

# Проверка, что метод next_int() возвращает целое значение в диапазоне [0, m)
def test_2():
    gen = quadratic_congruential_generator(seed=1, a=2, b=3, c=4, m=97)
    prng = MyPRNG(gen, m=97)
    val = prng.next_int()
    assert isinstance(val, int)
    assert 0 <= val < 97

# Проверка, что метод next_float() возвращает вещественное значение в диапазоне [0.0, 1.0)
def test_3():
    gen = quadratic_congruential_generator(seed=5, a=2, b=3, c=1, m=101)
    prng = MyPRNG(gen, m=101)
    val = prng.next_float()
    assert isinstance(val, float)
    assert 0.0 <= val < 1.0

# Проверка, что метод next_str() возвращает случайную строку заданной длины
def test_4():
    gen = quadratic_congruential_generator(seed=7, a=1, b=1, c=1, m=127)
    prng = MyPRNG(gen, m=127)
    s = prng.next_str(length=12)
    assert isinstance(s, str)
    assert len(s) == 12

# Проверка, что метод shuffle_str() возвращает перемешанную версию исходной строки
def test_5():
    gen = quadratic_congruential_generator(seed=42, a=3, b=2, c=1, m=211)
    prng = MyPRNG(gen, m=211)
    original = "abcdef"
    shuffled = prng.shuffle_str(original)
    assert isinstance(shuffled, str)
    assert sorted(shuffled) == sorted(original)
    assert shuffled != original
```

test_prg6pabN3149_subst.py

```python
import io
import sys
from types import SimpleNamespace
from prg6pabN3149_rnd import MyPRNG, quadratic_congruential_generator
from prg6pabN3149_subst import subst_float_args


def make_test_prng(seed=1, a=2, b=3, c=4, m=1000) -> MyPRNG:
    gen = quadratic_congruential_generator(seed=seed, a=a, b=b, c=c, m=m)
    return MyPRNG(gen, m=m)

# Тест проверяет работу декоратора @subst_float_args с позиционными float-аргументами
def test_1(monkeypatch):
    prng = make_test_prng()
    log_output = io.StringIO()
    monkeypatch.setattr(sys, "stderr", log_output)

    @subst_float_args(prng)
    def compute(x: float, y: float) -> float:
        return x + y

    result = compute(1.0, 2.0)
    assert isinstance(result, float)
    log = log_output.getvalue()
    assert "compute" in log
    assert "заменён на" in log

# Тест проверяет работу декоратора над функцией с именованными float-аргументами
def test_2(monkeypatch):
    prng = make_test_prng()
    log_output = io.StringIO()
    monkeypatch.setattr(sys, "stderr", log_output)

    @subst_float_args(prng)
    def area(length: float, width: float) -> float:
        return length * width

    result = area(length=4.2, width=3.1)
    assert isinstance(result, float)
    log = log_output.getvalue()
    assert "area" in log
    assert "заменён на" in log
```

```python
# Тест проверяет работу декоратора над классом — замена float-аргументов в методах
def test_3(monkeypatch):
    prng = make_test_prng()
    log_output = io.StringIO()
    monkeypatch.setattr(sys, "stderr", log_output)

    @subst_float_args(prng)
    class Calculator:
        def __init__(self, multiplier: float):
            self.multiplier = multiplier

        def scale(self, value: float) -> float:
            return self.multiplier * value

    c = Calculator(2.0)
    result = c.scale(10.0)
    assert isinstance(result, float)
    log = log_output.getvalue()
    assert "scale" in log
    assert "заменён на" in log

# Тест проверяет, что не float аргументы остаются без изменений при использовании декоратора
def test_4(monkeypatch):
    prng = make_test_prng()
    log_output = io.StringIO()
    monkeypatch.setattr(sys, "stderr", log_output)

    @subst_float_args(prng)
    def describe(x: float, label: str) -> str:
        return f"{label}: {x:.2f}"

    result = describe(3.0, "value")
    assert result.startswith("value:")
    log = log_output.getvalue()
    assert "заменён на" in log
    assert "describe" in log
```